Sesión 4. Primeros circuitos cuánticos con Qiskit

1. Introducción

Durante las tres primeras sesiones hemos trabajado directamente con el formalismo matemático de la computación cuántica:

- Vectores complejos para representar estados.
- Matrices para representar transformaciones.
- Productos matriciales para calcular la evolución.
- Módulos al cuadrado para obtener probabilidades.
- Simulaciones con NumPy para representar medidas.

Por ejemplo, el estado:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

puede representarse con NumPy:

```
import numpy as npket_0 = np.array(
    [1, 0],
    dtype=complex)
```

Y la transformación X:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

puede aplicarse mediante:

```
X = np.array([    [0, 1],    [1, 0]], dtype=complex)estado_final =
X @ ket_0
```

El resultado es:

$$X|0\rangle = |1\rangle$$

Qiskit permite describir este mismo proceso utilizando el modelo de circuitos:

```
from qiskit import QuantumCircuit
circuito = QuantumCircuit(1)
```

```
circuito.x(0)
```

El formalismo matemático no desaparece. Qiskit proporciona una abstracción para:

- Construir circuitos.
- Visualizarlos.
- Simular su evolución.
- Realizar medidas.
- Obtener resultados experimentales.
- Preparar los circuitos para ejecutarlos en computadores cuánticos.

Esta será la primera sesión en la que utilizaremos formalmente Qiskit.

2. El ecosistema de Qiskit

Qiskit es un conjunto de herramientas para construir, transformar y ejecutar programas cuánticos. El SDK trabaja principalmente en el nivel de circuitos cuánticos, operadores y primitivas de ejecución. Qiskit

En esta sesión utilizaremos dos componentes:

Qiskit SDK

Proporciona:

- QuantumCircuit.
- Puertas cuánticas.
- Registros cuánticos y clásicos.
- Statevector.
- Herramientas de visualización.
- Funciones de transpilación.

Qiskit Aer

Proporciona simuladores de alto rendimiento. Permite ejecutar circuitos ideales y, en sesiones posteriores, incorporar modelos de ruido. quantum.cloud.ibm.com

2.1. Instalación

Instalación básica

En una terminal:

```
pip install qiskit qiskit-aer matplotlib pylatexenc
```

Desde un notebook:

```
%pip install qiskit qiskit-aer matplotlib pylatexenc
```

Después de la instalación puede ser necesario reiniciar el kernel.

NOTA: pylatexenc se utiliza para algunas representaciones gráficas de circuitos.

Comprobación

```
import qiskit
import qiskit_aer
print("Qiskit:", qiskit.__version__)
print("Qiskit Aer:", qiskit_aer.__version__)
```

Es preferible no diseñar el código dependiendo de un número exacto de versión. Qiskit evoluciona y algunas interfaces pueden cambiar.

2.2. Importaciones de la sesión

import numpy as npimport matplotlib.pyplot as pltfrom qiskit import QuantumCircuitfrom qiskit import transpilefrom qiskit.quantum_info import Statevectorfrom qiskit_aer import AerSimulatorfrom qiskit.visualization import plot_histogram

Cada importación tiene una función diferente:

Elemento	Finalidad
QuantumCircuit	Construir circuitos
transpile	Adaptar y optimizar circuitos para un backend
Statevector	Representar estados puros mediante sus amplitudes
AerSimulator	Ejecutar circuitos simulados
plot_histogram	Representar conteos o distribuciones
NumPy	Comparar Qiskit con el formalismo matricial

3. Modelo de circuitos cuánticos

Un circuito cuántico describe una secuencia ordenada de operaciones sobre uno o varios qubits.

De forma esquemática:



Los principales elementos de un circuito son:

1. Qubits.
2. Líneas de qubit.
3. Puertas cuánticas.
4. Medidas.
5. Bits clásicos.
6. Orden temporal.

3.1. Estado inicial

Por defecto, los qubits de un circuito de Qiskit se inicializan en el estado:

$$|0\rangle$$

Un circuito de dos qubits comienza en:

$$|00\rangle$$

Uno de tres qubits comienza en:

$$|000\rangle$$

En general, un circuito de n qubits comienza en:

$$|0\rangle^{\otimes n} = |\underbrace{00\ldots0}_{n \text{ qubits}}\rangle$$

No es necesario aplicar ninguna operación para preparar este estado inicial.

3.2. Líneas de qubit

Cada línea horizontal del circuito representa la evolución temporal de un qubit.

Un circuito con dos qubits contiene dos líneas:

q_0:
q_1:

Las líneas no representan dos valores clásicos. Representan dos subsistemas cuánticos.

3.3. Puertas cuánticas

Las puertas representan transformaciones aplicadas a los qubits.

Por ejemplo:

```
circuito.x(0)
```

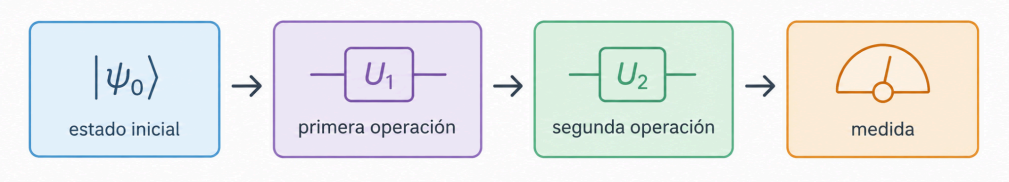
aplica la puerta X al qubit del índice 0.

Las puertas cuánticas que describen una evolución cerrada son transformaciones unitarias. Por ello:

- Conservan la norma.
- Son reversibles.
- No producen directamente resultados clásicos.

3.4. Orden temporal

Los circuitos de Qiskit se interpretan normalmente de izquierda a derecha:



estado inicial → primera operación → segunda operación → medida

Por ejemplo:

```
circuito.x(0)
circuito.h(0)
circuito.measure(0, 0)
```

significa:

1. Aplicar X.
2. Aplicar H.
3. Medir.

El orden es importante porque, en general:

$$AB \neq BA$$

En notación matricial, si primero se aplica A y después B, el estado final es:

$$|\psi_f\rangle = BA|\psi_0\rangle$$

La operación aplicada primero aparece más cerca del vector de estado.

4. Qubits y bits clásicos

Un qubit y un bit clásico son objetos distintos.

Qubit

Puede encontrarse en un estado:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Contiene amplitudes complejas y puede conservar información de fase.

Bit clásico

Solo puede almacenar: 0 ó 1.

En un circuito, los bits clásicos se utilizan principalmente para almacenar resultados de medida.

Medida

La medida conecta ambos tipos de información:

$$\text{qubit} \xrightarrow{\text{medida}} \text{bit clasico}$$

Una medida en la base computacional produce 0 o 1 y almacena el resultado en un bit clásico. Qiskit realiza por defecto estas medidas en la base computacional. IBM Quantum Documentation

5. Creación de un circuito

5.1. Circuito de un qubit

```
circuito = QuantumCircuit(1)
```

El argumento indica el número de qubits.

Podemos consultar sus propiedades:

```
print(circuito.num_qubits)
print(circuito.num_clbits)
```

Resultado:

```
1
0
```

El circuito tiene un qubit y ningún bit clásico.

5.2. Circuito con un qubit y un bit clásico

```
circuito = QuantumCircuit(1, 1)
```

Ahora:

```
print(circuito.num_qubits)
print(circuito.num_clbits)
```

Resultado:

```
1
1
```

La declaración del bit clásico no realiza ninguna medida. Únicamente reserva espacio para almacenar un resultado.

5.3. Circuito de varios qubits

```
circuito = QuantumCircuit(3, 3)
```

Este circuito contiene:

- Tres qubits.
- Tres bits clásicos.

Los índices comienzan en cero:

$$q_0,\ q_1,\ q_2$$
$$c_0,\ c_1,\ c_2$$

6. Visualización del circuito

La clase QuantumCircuit proporciona el método draw() para representar el circuito. quantum.cloud.ibm.com

6.1. Representación textual

```
circuito.draw()
```

También puede utilizarse:

```
print(circuito.draw())
```

La representación textual es útil porque:

- Funciona en terminales.
- No requiere una salida gráfica.
- Permite comprobar rápidamente el circuito.
- Es adecuada para depuración.

6.2. Representación con Matplotlib

```
circuito.draw("mpl")
```

En un notebook puede mostrarse mediante:

```
display(circuito.draw("mpl"))
```

Esta representación es más apropiada para:

- Material docente.
- Informes.
- Presentaciones.
- Circuitos de complejidad moderada.

7. Primer circuito: el estado |0⟩

Creamos un circuito vacío:

```
circuito_0 = QuantumCircuit(1)
circuito_0.draw("mpl")
```

No contiene ninguna puerta. El qubit permanece en su estado inicial:

$$|\psi\rangle = |0\rangle$$

Su representación vectorial es:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

El circuito vacío no significa que el circuito no tenga estado. Significa que no se ha realizado ninguna transformación sobre el estado inicial.

7.1. Obtención del vector de estado

Statevector representa un estado cuántico puro mediante sus amplitudes complejas.

Podemos obtener el estado generado por un circuito sin medidas mediante:

```
estado_0 = Statevector.from_instruction(circuito_0)
print(estado_0)
```

También podemos acceder al array:

```
print(estado_0.data)
```

Resultado:

```
[1.+0.j 0.+0.j]
```

Esto coincide con:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

7.2. Tipo y dimensión

```
print(type(estado_0))
print(type(estado_0.data))
print(estado_0.data.shape)
```

estado_0 es un objeto Statevector, mientras que estado_0.data es un array de NumPy.

Para un qubit contiene $2^1 = 2$ amplitudes.

Para dos qubits contiene $2^2 = 4$ amplitudes.

En general, para n qubits: 2^n amplitudes.

7.3. Amplitudes y probabilidades exactas

Podemos obtener las probabilidades mediante:

```
probabilidades_0 = estado_0.probabilities()
print(probabilidades_0)
```

Resultado:

```
[1. 0.]
```

También podemos comprobarlo manualmente:

```
probabilidades_numpy = (np.abs(estado_0.data) ** 2)
```

```
print(probabilidades_numpy)
```

Ambos resultados deben coincidir:

```
np.allclose(estado_0.probabilities(), np.abs(estado_0.data) ** 2)
```

7.4. Diccionario de probabilidades

```
estado_0.probabilities_dict()
```

Resultado:

```
{"0": 1.0}
```

El diccionario suele omitir los resultados cuya probabilidad es cero.

Debe distinguirse cuidadosamente entre:

```
estado.probabilities_dict()
```

y:

```
resultado.get_counts()
```

El primero devuelve probabilidades teóricas. El segundo devuelve conteos obtenidos mediante shots.

8. Preparación del estado |1⟩

Para transformar $|0\rangle$ en $|1\rangle$ utilizamos la puerta X.

Su matriz es:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Su acción es:

$$X|0\rangle = |1\rangle$$
$$X|1\rangle = |0\rangle$$

En Qiskit:

```
circuito_1 = QuantumCircuit(1)
circuito_1.x(0)
circuito_1.draw("mpl")
```

El argumento 0 indica el qubit sobre el que se aplica la puerta.

8.1. Vector de estado

```
estado_1 = Statevector.from_instruction(circuito_1)
print(estado_1.data)
```

Resultado:

```
[0.+0.j 1.+0.j]
```

Por tanto:

$$|\psi\rangle = |1\rangle$$

Las probabilidades son:

```
estado_1.probabilities()
```

Resultado:

```
[0. 1.]
```

Y:

```
estado_1.probabilities_dict()
```

produce:

```
{"1": 1.0}
```

9. Comparación con NumPy

9.1. Cálculo matricial

```
ket_0_numpy = np.array(    [1, 0],    dtype=complex)
X_numpy = np.array([    [0, 1],    [1, 0]], dtype=complex)
resultado_numpy = (    X_numpy @ ket_0_numpy)
print(resultado_numpy)
```

Resultado:

```
[0.+0.j 1.+0.j]
```

9.2. Circuito equivalente

```
circuito = QuantumCircuit(1)
```

```
circuito.x(0)
resultado_qiskit = (Statevector.from_instruction(circuito).data)
print(resultado_qiskit)
```

9.3. Comparación

```
np.allclose(resultado_numpy,resultado_qiskit)
```

```
Resultado:
```

```
True
```

Qiskit no aplica una física diferente. Representa el mismo proceso mediante el modelo de circuitos.

10. Dos aplicaciones de la puerta X

Como $X^2 = I$, aplicar X dos veces devuelve el estado inicial:

$$X(X|0\rangle) = |0\rangle$$

Creamos el circuito:

```
circuito_xx = QuantumCircuit(1)
circuito_xx.x(0)
circuito_xx.x(0)
circuito_xx.draw("mpl")
```

Obtenemos su estado:

```
estado_xx = Statevector.from_instruction(circuito_xx)
print(estado_xx.data)
```

Resultado esperado:

```
[1.+0.j 0.+0.j]
```

Comprobación:

```
np.allclose(estado_xx.data, ket_0_numpy)
```

11. Circuito, estado, medida y resultado

Es fundamental no confundir estos conceptos.

Concepto	Descripción	Ejemplo
Circuito	Secuencia de operaciones	QuantumCircuit(1)
Estado	Amplitudes del sistema	Statevector
Probabilidades	Módulos al cuadrado	[0.5, 0.5]
Medida	Operación cuántico-clásica	measure()
Shot	Una ejecución y medida	"0"
Conteos	Número de apariciones	{"0": 492, "1": 508}
Frecuencias	Conteos divididos entre shots	{"0": 0.492, "1": 0.508}

- Un circuito es una receta.
- Un estado es el resultado cuántico de aplicar esa receta hasta un momento determinado.
- Una medida produce un resultado clásico.
- Los conteos resumen muchas ejecuciones.

13. Añadir una medida

Creamos un circuito con un qubit y un bit clásico:

```
circuito_medida_0 = QuantumCircuit(1, 1)
```

Añadimos la medida:

```
circuito_medida_0.measure(0, 0)
```

Esto significa:

- Medir el qubit 0.
- Guardar el resultado en el bit clásico 0.

Visualización:

```
circuito_medida_0.draw("mpl")
```

13.1. Medida del estado $|1\rangle$

```
circuito_medida_1 = QuantumCircuit(1, 1)
circuito_medida_1.x(0)
circuito_medida_1.measure(0, 0)
circuito_medida_1.draw("mpl")
```

El circuito realiza:

$$|0\rangle \xrightarrow{X} |1\rangle \xrightarrow{\text{medida}} 1$$

El bit clásico almacenará 1 con probabilidad 1 en un simulador ideal.

13.2. measure() frente a measure_all()

Medida explícita

```
circuito.measure(qubit, bit_clasico)
```

Ejemplo:

```
circuito = QuantumCircuit(2, 2)
circuito.measure(0, 0)
circuito.measure(1, 1)
```

Permite controlar la correspondencia entre qubits y bits clásicos.

Medir todos los qubits

```
circuito.measure_all()
```

Este método añade las medidas y, si es necesario, los bits clásicos correspondientes.

Ejemplo:

```
circuito = QuantumCircuit(2)
circuito.measure_all()
```

Diferencia importante

QuantumCircuit(2, 2) crea bits clásicos, pero no añade medidas.

En cambio, circuito.measure_all() añade operaciones de medida.

13.3. Separar el circuito de estado y el circuito de medida

Es conveniente trabajar con dos circuitos relacionados.

Circuito sin medida

```
circuito_estado = QuantumCircuit(1)
circuito_estado.x(0)
```

Se utiliza para obtener el estado exacto:

```
estado = Statevector.from_instruction(circuito_estado)
```

Copia con medida

```
circuito_medida = circuito_estado.copy()
circuito_medida.measure_all()
```

Se utiliza para obtener resultados clásicos.

Este patrón evita mezclar dos objetivos:

- Inspeccionar las amplitudes.
- Simular el proceso de medida.

Las medidas no son puertas unitarias ordinarias. Por ello, Statevector.from_instruction() debe aplicarse al circuito unitario previo a la medida.

14. Simulación

14.1. Simulación exacta

La simulación mediante Statevector calcula exactamente las amplitudes del estado ideal.

```
estado = Statevector.from_instruction(circuito_estado)
```

No utiliza shots.

No produce un resultado aleatorio.

No devuelve conteos.

Proporciona:

- Amplitudes complejas.
- Probabilidades exactas.
- Información de fase.
- El estado completo del simulador ideal.

Por ejemplo:

```
print("Amplitudes:", estado.data)
print(    "Probabilidades:",    estado.probabilities())
```

14.2. Simulación mediante shots

Para simular ejecuciones con medida utilizaremos AerSimulator.

```
simulador = AerSimulator()
```

La documentación oficial de Aer recomienda transpilar el circuito para el simulador antes de ejecutarlo. Qiskit Aer 0.17.1

Flujo completo

```
circuito = QuantumCircuit(1, 1)
circuito.x(0)
circuito.measure(0, 0)
circuito_transpilado = transpile(circuito, simulador)
trabajo = simulador.run(circuito_transpilado, shots=1000,
seed_simulator=20)
resultado = trabajo.result()
conteos = resultado.get_counts()
print(conteos)
```

Resultado esperado:

```
{"1": 1000}
```

15. ¿Qué es la transpilación?

Un circuito puede expresarse mediante operaciones abstractas que no coinciden directamente con las instrucciones admitidas por un backend concreto.

La transpilación adapta el circuito al backend de destino.

```
circuito_transpilado = transpile(circuito, simulador)
```

Puede realizar tareas como:

- Descomponer puertas.
- Adaptar el circuito al conjunto de instrucciones disponible.
- Reorganizar operaciones.
- Optimizar el circuito.
- Mapear qubits lógicos a qubits físicos.

En esta sesión utilizaremos un simulador ideal y circuitos muy sencillos, por lo que el circuito transpilado puede parecer idéntico al original. Sin embargo, la separación entre construcción y transpilación será fundamental cuando se trabaje con hardware real.

15.1. Trabajos y resultados

La llamada trabajo = simulador.run(...) devuelve un objeto que representa el trabajo de ejecución.

El resultado se recupera mediante:

```
resultado = trabajo.result()
```

Los conteos se obtienen con:

```
conteos = resultado.get_counts()
```

El flujo completo es:



Aunque en un simulador local el trabajo suele completarse rápidamente, esta estructura también permite representar ejecuciones que requieren tiempo.

16. Número de shots

Un shot representa una preparación y ejecución completa del circuito seguida de una medida.

Si usamos shots=1000. se simulan 1.000 repeticiones independientes.

No significa que se mida 1.000 veces el mismo qubit después de haber colapsado.

Cada shot representa:

1. Inicializar los qubits.
2. Aplicar las operaciones.
3. Realizar la medida.
4. Registrar el resultado.

16.1. Estados deterministas

Para $|0\rangle$: {"0": 1000}

Para $|1\rangle$: {"1": 1000}

En un simulador ideal no hay fluctuación porque las probabilidades son exactamente 0 y 1.

16.2. Estados probabilísticos

Para un estado con:

$$P(0) = P(1) = \frac{1}{2}$$

podrían obtenerse: {"0": 493, "1": 507}.

No es obligatorio obtener exactamente 500 y 500.

17. Semilla del simulador

Podemos fijar:

```
seed_simulator=25
```

La semilla permite reproducir la misma secuencia pseudoaleatoria:

```
trabajo = simulador.run(circuito_transpilado, shots=1000,
seed_simulator=25)
```

La semilla:

- No elimina la naturaleza probabilística del modelo.
- No modifica las probabilidades teóricas.
- Permite reproducir los mismos conteos en el simulador.
- Resulta útil para prácticas, pruebas y depuración.

En hardware real no se controla el proceso físico mediante una semilla de este tipo.

18. Conteos y frecuencias

Supongamos: conteos = { "0": 487, "1": 513} con: shots = 1000

Las frecuencias son:

$$f(0) = \frac{487}{1000} = 0,487$$

$$f(1) = \frac{513}{1000} = 0,513$$

En Python:

```
frecuencias = {
    resultado: conteo / shots
```

```
    for resultado, conteo in conteos.items()
}
print(frecuencias)
```

Las frecuencias experimentales aproximan las probabilidades teóricas:

$$f(x) \longrightarrow P(x)$$

cuando aumenta el número de shots.

18.1. Probabilidades exactas frente a frecuencias

Consideremos un estado con:

$$P(0) = 0,5$$

$$P(1) = 0,5$$

Simulación exacta

```
estado.probabilities_dict()
```

podría producir:

```
{    "0": 0.5,    "1": 0.5}
```

Muestreo

```
resultado.get_counts()
```

podría producir:

```
{    "0": 493,    "1": 507}
```

Con 1.000 shots, las frecuencias serían:

```
{    "0": 0.493,    "1": 0.507}
```

Propiedad	Probabilidad exacta	Frecuencia
Procede del modelo matemático	Sí	No directamente
Se obtiene mediante shots	No	Sí
Presenta fluctuación estadística	No	Sí
Suma 1	Sí	Sí, tras dividir por shots

Contiene fase	No por sí sola	No

19. Representación mediante histogramas

Qiskit proporciona:

```
plot_histogram(conteos)
```

También puede personalizarse:

```
plot_histogram(conteos,title="Resultados de la medida",
color="#4472C4")
```

En un notebook:

```
figura = plot_histogram(conteos,title="Resultados")
display(figura)
```

El histograma representa:

- Resultados clásicos.
- Conteos o probabilidades.
- La distribución observada.

No representa directamente:

- Amplitudes.
- Fases.
- El vector de estado.
- Las operaciones del circuito.
- La trayectoria individual de cada qubit.

20. Primera superposición en Qiskit

Aunque las puertas se estudiarán formalmente en la siguiente sesión, podemos utilizar la puerta H para comparar simulación exacta y muestreo.

circuito_superposicion = QuantumCircuit(1)
circuito_superposicion.h(0)
estado_superposicion = (Statevector.from_instruction(circuito_superposicion))
print(estado_superposicion.data)

Resultado:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Vector:

```
[0.70710678+0.j 0.70710678+0.j]
```

Probabilidades exactas:

```
estado_superposicion.probabilities_dict()
```

Resultado:

```
{    "0": 0.5,    "1": 0.5}
```

20.1. Medida de la superposición

```
circuito_superposicion_medida = (circuito_superposicion.copy())
circuito_superposicion_medida.measure_all()
circuito_transpilado = transpile(
    circuito_superposicion_medida,
    simulador
)
resultado = simulador.run(
    circuito_transpilado,
    shots=1000,
    seed_simulator=42
).result()
conteos = resultado.get_counts()
print(conteos)
```

Los conteos deberían aproximarse a:

```
{    "0": 500,    "1": 500}
```

No tienen por qué coincidir exactamente.

21. Comparación de diferentes shots

Podemos estudiar:

```
shots_estudiados = [    10,    100,    1_000,    10_000]
for shots in shots_estudiados:
    resultado = simulador.run(
        circuito_transpilado,
        shots=shots,
        seed_simulator=42
```

```
    ).result()
    conteos = resultado.get_counts()
    frecuencia_0 = (
        conteos.get("0", 0) / shots
    )
    frecuencia_1 = (
        conteos.get("1", 0) / shots
    )
    print(
        f"shots={shots:>6} "
        f"f(0)={frecuencia_0:.4f} "
        f"f(1)={frecuencia_1:.4f}"
    )
```

Debe utilizarse conteos.get("0", 0) en lugar de conteos["0"] porque un resultado con frecuencia cero podría no aparecer como clave.

22. Circuitos de varios qubits

Creamos un circuito de dos qubits:

```
circuito_2q = QuantumCircuit(2)circuito_2q.draw("mpl")
```

El estado inicial es:

$$|00\rangle$$

El vector de estado contiene cuatro amplitudes:

```
estado_2q = Statevector.from_instruction(circuito_2q)
print(estado_2q.data)
```

Resultado:

```
[1.+0.j 0.+0.j 0.+0.j 0.+0.j]
```

Esto representa:

$$1|00\rangle + 0|01\rangle + 0|10\rangle + 0|11\rangle$$

22.1. Preparación de estados de dos qubits

Estado $|00\rangle$:

```
circuito_00 = QuantumCircuit(2)
```

Aplicar X al qubit 0:

```
circuito_01 = QuantumCircuit(2)
circuito_01.x(0)
```

Aplicar X al qubit 1:

```
circuito_10 = QuantumCircuit(2)
circuito_10.x(1)
```

Aplicar X a ambos:

```
circuito_11 = QuantumCircuit(2)
circuito_11.x(0)
circuito_11.x(1)
```

22.2. Orden de bits en Qiskit

El orden de bits es una de las fuentes de errores más frecuentes.

Qiskit numera los qubits comenzando por 0, pero las cadenas se muestran normalmente con el bit de índice más alto a la izquierda. En una cadena de dos bits, el orden visual es $q_1 q_0$; por eso, si $q_0 = 1$ y $q_1 = 0$, la cadena mostrada es "01". IBM Quantum Documentation

Ejemplo

```
circuito = QuantumCircuit(2)
circuito.x(0)
estado = Statevector.from_instruction(circuito)
print(estado.probabilities_dict())
```

Resultado:

```
{"01": 1.0}
```

Esto significa:

$$q_1 = 0$$
$$q_0 = 1$$

Por tanto:

$$|q_1 q_0\rangle = |01\rangle$$

Tabla de estados

Operaciones	q_1	q_0	**Cadena mostrada**

Ninguna	0	0	"00"
x(0)	0	1	"01"
x(1)	1	0	"10"
x(0) y x(1)	1	1	"11"

22.3. Orden del vector de estado

Para dos qubits, las amplitudes aparecen en el orden:

$$|00\rangle,\ |01\rangle,\ |10\rangle,\ |11\rangle$$

El vector:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$$

se almacena como:

$$\begin{pmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{pmatrix}$$

x(0)

```
circuito = QuantumCircuit(2)
circuito.x(0)
Statevector.from_instruction(circuito).data
```

Resultado:

$$\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

Corresponde a $|01\rangle$

x(1)

```
circuito = QuantumCircuit(2)
circuito.x(1)
Statevector.from_instruction(circuito).data
```

Resultado:

$$\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Corresponde a $|10\rangle$

22.4. Orden de las medidas

Consideremos:

```
circuito = QuantumCircuit(2, 2)
circuito.x(0)
circuito.measure(0, 0)
circuito.measure(1, 1)
```

El resultado mostrado será:

```
01
```

porque los bits clásicos se muestran como:

$$c_1 c_0$$

Si intercambiamos el destino de las medidas:

```
circuito.measure(0, 1)
circuito.measure(1, 0)
```

la cadena clásica cambia, aunque el estado cuántico anterior a la medida sea el mismo.

Esto demuestra que deben diferenciarse:

- Orden de los qubits.
- Orden de las amplitudes.
- Correspondencia entre qubits y bits clásicos.
- Presentación final de las cadenas.

22.5. Circuito de dos qubits con medidas

```
circuito = QuantumCircuit(2, 2)
circuito.x(0)
circuito.measure(0, 0)
circuito.measure(1, 1)
circuito.draw("mpl")
```

Ejecución:

```
simulador = AerSimulator()
circuito_transpilado = transpile(circuito, simulador)
resultado = simulador.run(circuito_transpilado, shots=1000,
seed_simulator=25).result()
conteos = resultado.get_counts()
print(conteos)
```

Resultado:

```
{"01": 1000}
```

23. Uso de barreras

Una barrera puede añadirse mediante:

```
circuito.barrier()
```

Ejemplo:

```
circuito = QuantumCircuit(1, 1)
circuito.x(0)
circuito.barrier()
circuito.measure(0, 0)
```

La barrera:

- Ayuda a separar visualmente etapas.
- Puede restringir algunas reorganizaciones del transpilador.
- No modifica el estado por sí misma.
- No es una puerta física que transforme amplitudes.

En materiales docentes puede utilizarse para distinguir:

1. Preparación.
2. Transformación.
3. Medida.

No conviene añadir barreras innecesarias, porque pueden limitar optimizaciones posteriores.

24. Funciones auxiliares

24.1. Función auxiliar para ejecutar circuitos

Para evitar repetir código podemos definir:

```
def ejecutar_con_shots(
    circuito,
    shots=1000,
    semilla=42):
    if not isinstance(shots, int) or isinstance(shots, bool):
        raise TypeError(
            "shots debe ser un entero"
        )
    if shots <= 0:
        raise ValueError(
            "shots debe ser positivo"
        )
    simulador = AerSimulator()
    circuito_transpilado = transpile(
        circuito,
        simulador
    )
    resultado = simulador.run(
        circuito_transpilado,
        shots=shots,
        seed_simulator=semilla
    ).result()
    return resultado.get_counts()
```

Uso:

```
circuito = QuantumCircuit(1, 1)
circuito.x(0)
circuito.measure(0, 0)
conteos = ejecutar_con_shots(circuito, shots=1000, semilla=20)
print(conteos)
```

24.2. Función auxiliar para calcular frecuencias

```
def calcular_frecuencias(
    conteos,
    shots=None):
    if shots is None:
        shots = sum(conteos.values())
    if shots <= 0:
```

```
        raise ValueError(
            "El número de shots debe ser positivo"
        )
    return {
        resultado: conteo / shots
        for resultado, conteo
        in conteos.items()
    }
```

Uso:

```
frecuencias = calcular_frecuencias(conteos)
print(frecuencias)
```

Es preferible calcular el total a partir de los conteos cuando no existe una razón específica para proporcionarlo por separado:

```
sum(conteos.values())
```

25. Flujo de trabajo recomendado

Un flujo útil para las próximas sesiones será:

Paso 1. Construir el circuito

```
circuito = QuantumCircuit(1)
```

Paso 2. Añadir puertas

```
circuito.x(0)
```

Paso 3. Visualizarlo

```
circuito.draw("mpl")
```

Paso 4. Obtener el estado exacto

```
estado = Statevector.from_instruction(    circuito)
```

Paso 5. Crear una copia para medir

```
circuito_medida = circuito.copy()
circuito_medida.measure_all()
```

Paso 6. Transpilar

```
circuito_transpilado = transpile(circuito_medida, simulador)
```

Paso 7. Ejecutar shots

```
resultado = simulador.run(circuito_transpilado,
shots=1000).result()
```

Paso 8. Obtener conteos

```
conteos = resultado.get_counts()
```

Paso 9. Representar el histograma

```
plot_histogram(conteos)
```

Paso 10. Comparar

Comparar:

- Vector exacto.
- Probabilidades teóricas.
- Conteos.
- Frecuencias.
- Predicción matemática.

26. Ejemplo completo

```
import numpy as np from qiskit
import QuantumCircuit from qiskit
import transpile from qiskit.quantum_info
import Statevector from qiskit_aer
import AerSimulator from qiskit.visualization
import plot_histogram

# 1. Construcción
circuito = QuantumCircuit(1)

# 2. Operación
circuito.x(0)

# 3. Visualización
display(circuito.draw("mpl"))
```

```
# 4. Estado exacto
estado = Statevector.from_instruction(circuito)
print("Vector de estado:")
print(estado.data)
print("Probabilidades:")
print(estado.probabilities_dict())

# 5. Copia con medidas
circuito_medida = circuito.copy()
circuito_medida.measure_all()
display(circuito_medida.draw("mpl"))

# 6. Simulador
```

27. Comparación entre NumPy y Qiskit

Operación	NumPy	Qiskit
Estado inicial	Array complejo	Estado inicial del circuito
Transformación	Matriz	Puerta
Evolución	Producto matricial	Aplicación de puertas
Estado final	Array	Statevector
Probabilidades	np.abs(estado)**2	state.probabilities()
Medida	Simulación manual	Instrucción measure
Repeticiones	rng.choice()	Shots del backend
Resultados	Array/conteos	get_counts()
Visualización	Matplotlib	draw() y plot_histogram()

Qiskit automatiza parte del trabajo, pero comprender NumPy sigue siendo importante para:

- Verificar resultados.
- Entender las puertas.
- Analizar fases.
- Detectar errores de orden.
- Relacionar circuitos y álgebra lineal.

28. Limitaciones del simulador ideal

En esta sesión utilizaremos simulación ideal.

Un simulador ideal no incorpora automáticamente:

- Decoherencia.
- Errores de puerta.
- Errores de medida.
- Conectividad física limitada.
- Duración de las puertas.
- Calibraciones.
- Deriva temporal del dispositivo.
- Interacción no deseada entre qubits.

Por tanto, que un circuito funcione perfectamente en un Statevector o en un AerSimulator ideal no implica que produzca exactamente el mismo resultado en un computador cuántico real.

44. Errores frecuentes

Error 1. Confundir circuito y ejecución

```
circuito = QuantumCircuit(1)
```

solo construye una descripción. No ejecuta nada.

Error 2. Confundir circuito y estado

Un circuito contiene operaciones. El estado es el vector resultante de aplicarlas.

```
estado = Statevector.from_instruction(circuito)
```

Error 3. Pensar que un bit clásico es un qubit

El qubit puede contener amplitudes complejas. El bit clásico almacena un resultado 0 o 1.

Error 4. Pensar que declarar bits clásicos mide

```
QuantumCircuit(1, 1)
```

no añade ninguna medida.

Error 5. Olvidar añadir medidas

Para obtener conteos clásicos deben existir instrucciones de medida.

Error 6. Obtener el vector después de medir

Para estudiar amplitudes es preferible conservar una versión del circuito sin medidas.

Error 7. Confundir probabilidades con conteos

`estado.probabilities_dict()` devuelve probabilidades.

`resultado.get_counts()` devuelve cantidades enteras observadas.

Error 8. Pensar que los shots calculan amplitudes

Los shots generan resultados de medida. No reconstruyen automáticamente el vector de estado.

Error 9. Esperar conteos exactos en un estado probabilístico

Con 1.000 shots y probabilidades 50/50 no tienen que aparecer exactamente 500 resultados de cada tipo.

Error 10. Acceder directamente a una clave ausente

Esto puede fallar: `conteos["0"]`

Es más seguro: `conteos.get("0", 0)`

Error 11. Interpretar mal el orden de bits

En Qiskit, para dos qubits, "01" significa:

$$q_1 = 0, q_0 = 1$$

Error 12. Omitir la transpilación

Para una ejecución robusta debe prepararse el circuito para el backend:

`transpile(circuito, simulador)`

Error 13. Creer que draw() muestra el estado

draw() muestra el circuito, no las amplitudes ni el resultado de ejecutarlo.

Error 14. Creer que el histograma muestra amplitudes

Un histograma de medidas muestra conteos o frecuencias clásicas. No muestra signos ni fases.